

Title: *"RabbitMQ and OpenStack Overview".*

Name: Abdulrahman Odhaib

Date: 2/11/2024

Student No.210201879

Contents:

1. Brief Overview of **OpenStack** and **RabbitMQ**
2. Brief Overview of **RabbitMQ**
3. How to **setup RabbitMQ**
4. **Taxi App Example** Using RabbitMQ

Brief Overview of **OpenStack** and RabbitMQ

OpenStack Overview:

- OpenStack is an open-source cloud computing platform that manages large pools of compute, storage, and networking resources.
- It consists of various components that can be scaled independently based on needs:
 - Nova: Manages and provisions compute resources.
 - Neutron: Handles networking.
 - Cinder: Manages block storage.
- Scalability Advantage: You can add more resources, like CPU or RAM, to existing nodes without having to redeploy the entire system.

2. Setting Up OpenStack:

- OpenStack can be deployed on various devices, such as laptops for initial testing and powerful servers for production.
- To increase specific resources like RAM or storage:
 - Add more compute nodes to scale the environment.
- Nodes: A node is a physical or virtual machine that provides resources for OpenStack services.

3. Accessing OpenStack via SSH (if you want):

- Ensure you have the correct IP and credentials.
- Use `ssh <username>@<ip-address>` to connect if you are on the same network.

Brief Overview of **RabbitMQ**

1. What is RabbitMQ?

- RabbitMQ is a message broker, not a traditional database like SQL.
- It facilitates communication between applications by sending and receiving messages in a reliable way.

2. Key Components of RabbitMQ:

- **Producer:** Sends messages to RabbitMQ.
- **Exchange:** Routes messages to appropriate queues based on defined rules.
- **Queue:** Stores messages until they are processed by consumers.
- **Consumer:** Retrieves and processes messages from the queue.

3. Use Cases of RabbitMQ:

- **Decoupling Services:** Asynchronous tasks like sending emails or processing images.
- **Event-Driven Systems:** Efficient handling of events like user sign-ups or order processing.

4. Troubleshooting RabbitMQ:

- If RabbitMQ fails to start, check service status: `sudo systemctl status rabbitmq-server`
- View logs in `/var/log/rabbitmq/` for more details.
- Common issues include configuration errors or resource limitations.

5. Managing RabbitMQ Users:

- Default credentials: `guest` / `guest` (for localhost access only).

- List users: `sudo rabbitmqctl list_users`
- Reset user password: `sudo rabbitmqctl change_password <username> <newpassword>`

Command to Download and configure RabbitMQ

1.Install RabbitMQ

- For Ubuntu
`sudo apt update`
`sudo apt install rabbitmq – server`
- For CentOS/Rocky Linux
`sudo yum install rabbitmq – server`

2. Start and Enable RabbitMQ

```
sudo systemctl start rabbitmq – server
sudo systemctl start rabbitmq – server
```

3.Enable the RabbitMQ Management Plugin (Optional)

- `sudo rabbitmq – plugins enable rabbitmq_management`

Note:The management interface will be available at

`http://<your_rabbitmq_ip>:15672`. The default username and password are both *guest*

If you want to create user just for OpenStack (it is better) follow these steps

- `sudo rabbitmqctl add_user openstack yourpassword`
- `sudo rabbitmqctl set_user_tags openstack administrator`
- `sudo rabbitmqctl set_permissions -p / openstack ".*" ".*" ".*"`

Lastly configure the `/etc/rabbitmq/rabbitmq.conf` (delete it if you faced any errors)

```
# /etc/rabbitmq/rabbitmq.conf  
loopback_users.guest = false # Allow guest access from remote
```

4. Restart RabbitMQ:

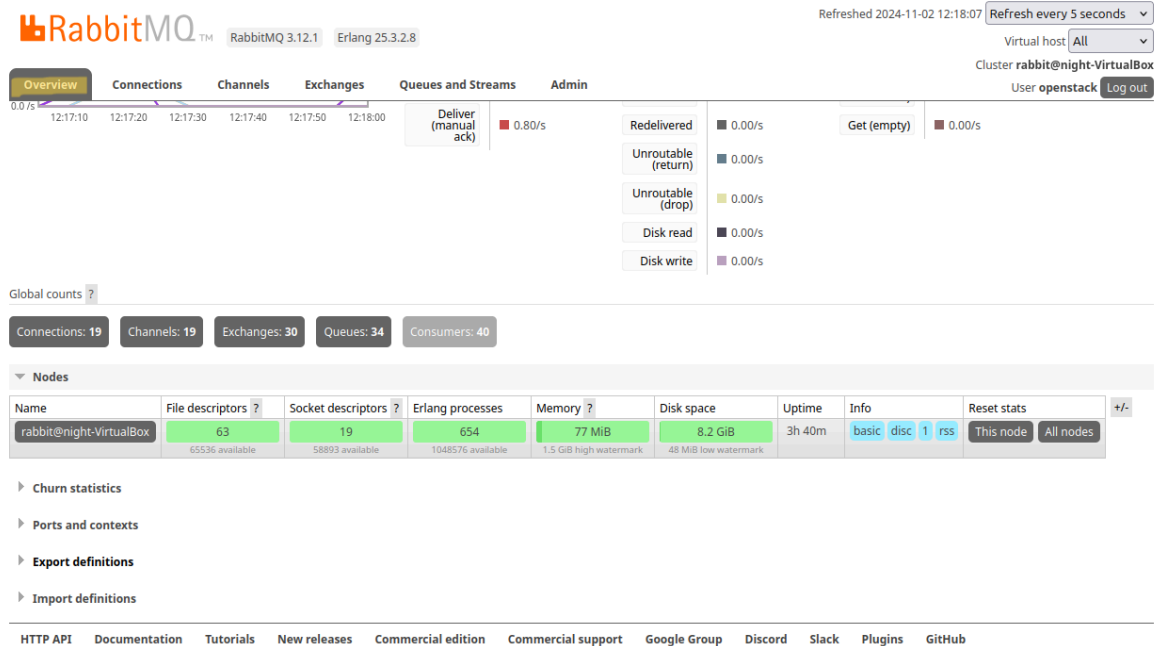
```
sudo systemctl restart rabbitmq -- server
```

Configure OpenStack Components to Use RabbitMQ:

- You will need to configure each OpenStack service to use RabbitMQ. This is done in the service's configuration file (usually found in `/etc/<service_name>/<service_name>.conf`).
- Here's an example for configuring the Nova service (`/etc/nova/nova.conf`):

```
transport_url = rabbit://openstack:yourpassword@controller
```

now you should be able to open RabbitMQ in the browser



Purpose: Provides a summary of the current state of RabbitMQ.

Information Displayed:

- General statistics about the server, such as the number of connections, channels, queues, and exchanges.
- Memory usage, message rates, and node status.

Usage: Useful for quickly assessing the health and performance of your RabbitMQ server.

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.trace	topic	D i			
/	amq.topic	topic	D			
/	cinder-backup_fanout	fanout	AD	0.00/s		
/	cinder-scheduler_fanout	fanout	AD	0.00/s	0.00/s	
/	cinder-volume.night-VirtualBox@lvmdriver-1_fanout	fanout	AD			
/	cinder-volume_fanout	fanout	AD	0.00/s		
/	conductor_fanout	fanout	AD			
/	homework	direct	D	0.00/s	0.00/s	
/	neutron	topic				
/	nova	topic				
/	openstack	topic		0.00/s	0.00/s	
/	q-plugin_fanout	fanout	AD			
/	q-reports-plugin_fanout	fanout	AD			
/	q-server-resource-versions_fanout	fanout	AD			


RabbitMQ 3.12.1 Erlang 25.3.2.8

Refreshed 2024-11-02 12:19:19 Refresh every 5 seconds

Virtual host All

Cluster rabbit@night-VirtualBox

User openstack Log out

Overview Connections Channels Exchanges Queues and Streams Admin

/	q-reports-plugin_fanout	fanout	AD			
/	q-server-resource-versions_fanout	fanout	AD			
/	scheduler_fanout	fanout	AD			
nova_cell1	(AMQP default)	direct	D	0.00/s	0.00/s	
nova_cell1	amq.direct	direct	D			
nova_cell1	amq.fanout	fanout	D			
nova_cell1	amq.headers	headers	D			
nova_cell1	amq.match	headers	D			
nova_cell1	amq.rabbitmq.trace	topic	D i			
nova_cell1	amq.topic	topic	D			
nova_cell1	compute_fanout	fanout	AD			
nova_cell1	conductor_fanout	fanout	AD			
nova_cell1	nova	topic		0.00/s	0.00/s	

Add a new exchange

Virtual host: /

Name:

Type: direct

Durability: Durable

Auto delete: No

Internal: No

Purpose: Manages the message routing rules.

Information Displayed:

- A list of all exchanges and their types (e.g., direct, fanout, topic, headers).
- Configuration details, bindings (how messages are routed), and the ability to create or delete exchanges.

Usage: Configure how messages should be distributed to the appropriate queues.

Queues

▼

All queues (34)

Pagination

Page 1 of 1

- Filter:

☐ Regex ?

Displaying 34 items , page size up to: 100

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	cinder-scheduler	classic		idle	0	0	0	0.00/s	0.00/s	0.00/s	
/	cinder-scheduler.night-VirtualBox	classic		idle	0	0	0				
/	cinder-scheduler_fanout_31f6e1c79f6a4b9f86d3115272679ae7	classic	Exp	idle	0	0	0	0.00/s	0.00/s	0.00/s	
/	cinder-volume	classic		idle	0	0	0				
/	cinder-volume.night-VirtualBox@lvmdriver-1	classic		idle	0	0	0				
/	cinder-volume.night-VirtualBox@lvmdriver-1.night-VirtualBox	classic		idle	0	0	0				
/	cinder-volume.night-VirtualBox@lvmdriver-1_fanout_b1b4e2fb08774722b2e5c796b88e1f69	classic	Exp	idle	0	0	0				
/	cinder-volume_fanout_fa2359e517934c939937cdab0da74433	classic	Exp	idle	0	0	0				
/	conductor	classic		idle	0	0	0				
/	conductor.night-VirtualBox	classic		idle	0	0	0				
/	conductor_fanout_45f140cb47c3492fba701e7ad730a72e	classic	Exp	idle	0	0	0				
/	conductor_fanout_b603638c368f484c8d59007255146f37	classic	Exp	idle	0	0	0				
/	hadoop-kli...	classic	Exp	idle	0	0	0	0.00/s	0.00/s	0.00/s	



RabbitMQ 3.12.1

Erlang 25.3.2.8

Refreshed 2024-11-02 12:22:01

Do not refresh

Virtual host All

Cluster rabbit@night-VirtualBox

User openstack Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queued messages last hour ?

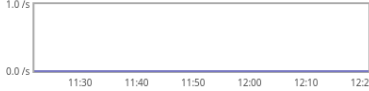


Ready 2

Unacked 0

Total 2

Message rates last hour ?



Publish 0.00/s

Deliver (manual ack) 0.00/s

Deliver (auto ack) 0.00/s

Consumer ack 0.00/s

Redelivered 0.00/s

Get (manual ack) 0.00/s

Get (auto ack) 0.00/s

Get (empty) 0.00/s

Details		State		Consumers							
Features	durable: true queue storage version: 1	State	idle	Consumers	0	Consumer capacity	0%				
Policy								Total	Ready	Unacked	
Operator policy								2	2	0	
Effective policy definition								In memory	Persistent	Transient, Paged Out	
								1	2	0	
								46 B	46 B	0 B	
								46 B	46 B	0 B	
								13 KiB			

Consumers (0)

Purpose: Manages and monitors message queues.

Information Displayed:

- A list of all queues, message counts, and consumer details.
- Options to purge (delete all messages in) or delete a queue, and view messages in real time.

Usage: Useful for observing the flow of messages, troubleshooting message buildup, and managing queue properties.

RabbitMQ™

RabbitMQ 3.12.1

Erlang 25.3.2.8

Refreshed 2024-11-02 12:22:55

Do not refresh

Virtual hostAll

Cluster rabbit@night-VirtualBox

User openstackLog out

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Users

All users (2)

Pagination

Page1of 1 - Filter:

Regex?

Displaying 2 items , page size up to: 100

Name	Tags	Can access virtual hosts	Has password
openstack	administrator	/	•
stackrabbit		/, nova_cell1	•

?
Add a user

Users

Virtual Hosts

Feature Flags

Policies

Limits

Cluster

HTTP APIDocumentationTutorialsNew releasesCommercial editionCommercial supportGoogle GroupDiscordSlackPluginsGitHub

Purpose: Configures users, permissions, and virtual hosts.

Information Displayed:

- User management (adding, deleting, or modifying users and setting permissions).
- Managing virtual hosts, which are logical divisions that allow different applications to use RabbitMQ without interfering with each other.

Usage: Essential for setting up secure access and managing how applications interact with RabbitMQ.

Taxi App Example Using RabbitMQ

Imagine a taxi booking app that processes different types of requests, such as ride bookings, payment processing, and notifications. RabbitMQ can help route these requests efficiently.

1. Ride Booking Request (Direct Exchange):

- **Scenario:** When a user books a ride, a message needs to be sent to the appropriate service for handling the booking.
 - **Exchange Type:** *Direct Exchange*
 - **How It Works:** A direct exchange routes messages to queues based on an exact match between the message's routing key and the queue's binding key.
 - **Example:**
 - A message with the routing key `ride.booking` is sent.
 - The exchange checks which queue is bound with the `ride.booking` key and routes the message to the `RideBookingQueue`.
-

2. Payment Processing (Topic Exchange):

- **Scenario:** The app needs to handle different types of payments (e.g., credit card, PayPal, and cash).
 - **Exchange Type:** *Topic Exchange*
 - **How It Works:** A topic exchange routes messages to one or more queues based on pattern matching of the routing key.
 - **Example:**
 - Routing keys like `payment.creditcard`, `payment.paypal`, and `payment.cash` are used.
 - The `PaymentProcessingQueue` subscribes to `payment.*`, receiving all payment-related messages.
 - Alternatively, specific queues can subscribe to detailed patterns (e.g., `payment.creditcard` for credit card transactions only).
-

3. Notifications (Fanout Exchange):

- **Scenario:** Sending notifications to all connected users, like promotional messages or alerts about ride availability.
 - **Exchange Type:** *Fanout Exchange*
 - **How It Works:** A fanout exchange routes messages to all bound queues, regardless of the routing key.
 - **Example:**
 - When a new promotion is launched, a message is sent to the fanout exchange.
 - All queues bound to this exchange (e.g., `UserNotificationQueue`, `DriverNotificationQueue`) receive the message and distribute notifications accordingly.
-

4. Header-Based Routing (Headers Exchange):

- **Scenario:** Processing requests that require custom matching based on message headers rather than routing keys.
- **Exchange Type:** *Headers Exchange*
- **How It Works:** A headers exchange routes messages based on header values instead of the routing key.
- **Example:**
 - A message for a high-priority ride might include headers like `type=ride` and `priority=high`.
 - The `HighPriorityRideQueue` is configured to receive messages where `priority=high`.

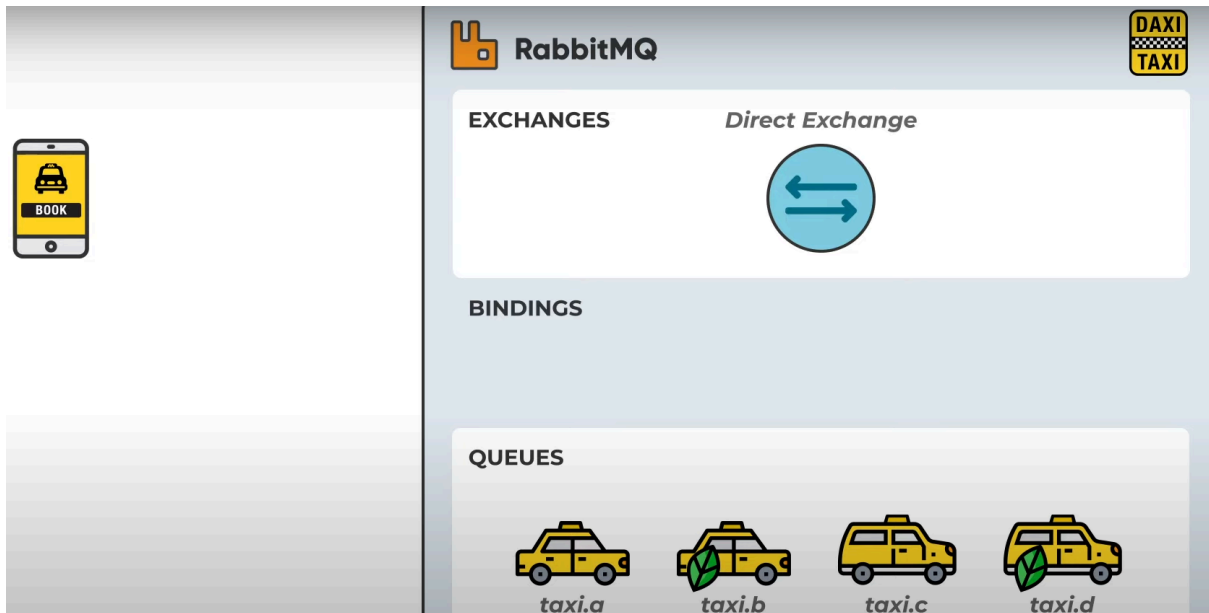
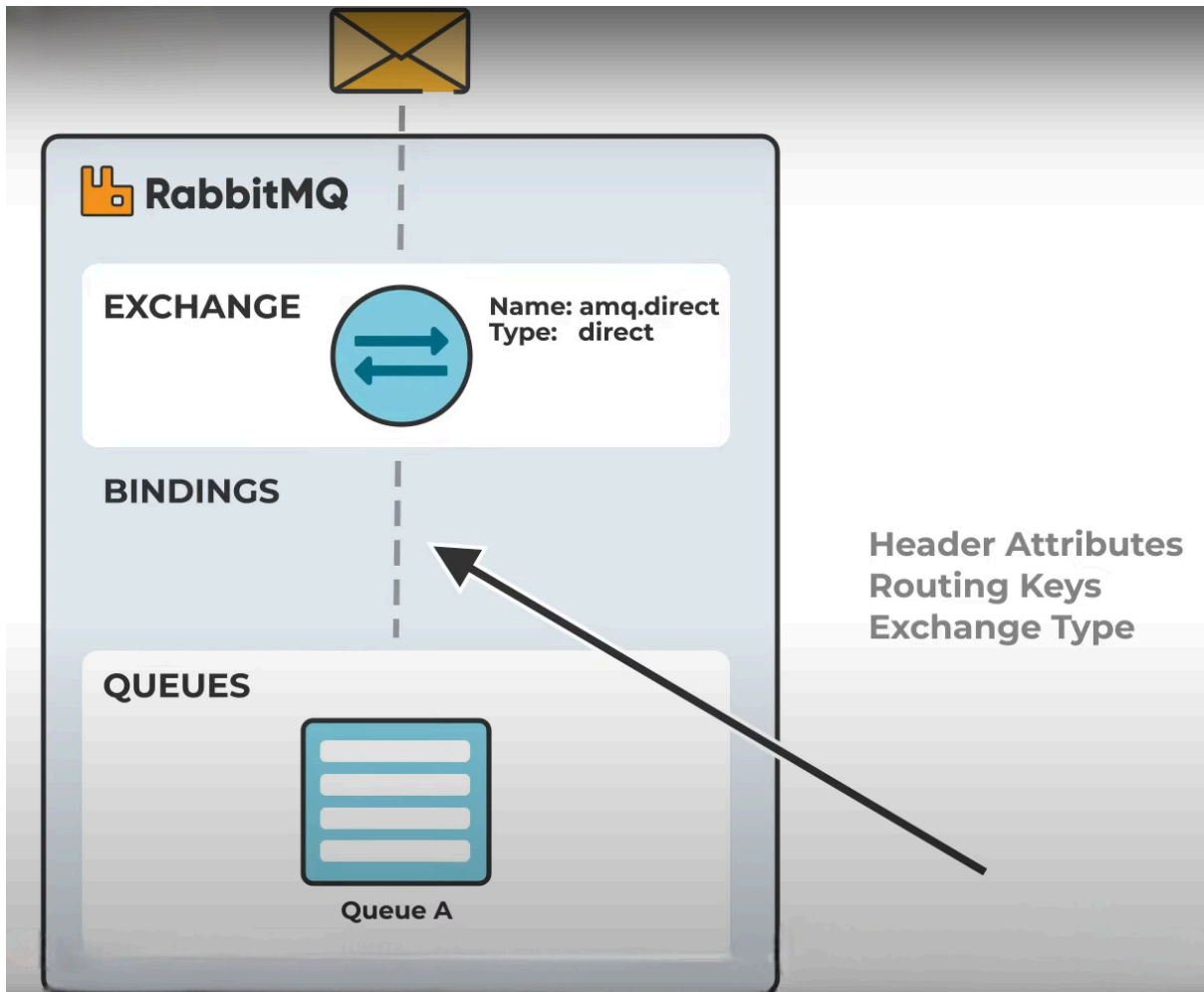
To summarize

Direct Exchange: Used for routing messages with exact matches, like ride bookings.

Topic Exchange: Useful for handling messages that require flexible pattern matching, like different types of payments.

Fanout Exchange: Ideal for broadcasting messages to multiple services, like sending notifications to users and drivers.

Headers Exchange: Provides advanced message routing based on custom headers, useful for prioritizing certain requests.



CloudAMQP